databricks

AI System Monitoring

LECTURE

AI Application Monitoring

In this lecture AI Application Monitoring, we discuss monitoring AI systems, Databricks Lakehouse Monitoring, Unity Catalog integration, monitoring model responses, and Lakehouse Monitoring workflows.

Monitoring AI Systems

Continuous logging and review of key component/system metrics

Why?

Used to help diagnose issues before they become **severe** or **costly**

Data to Monitor

Input data (tricky with existing models)
Data in vector databases/knowledge bases
Human feedback data
Prompt/queries and responses (legality)

AI Assets to Monitor

Mid-training checkpoints for analysis
Component evaluation metrics
AI system evaluation metrics
Performance/cost details

The aim of monitoring is to be as proactive as possible, rather than merely reacting to issues after they arise—which is often easier said than done. That's why features like the inference table and logging capabilities exist: they help you stay ahead and handle potential problems before they escalate.

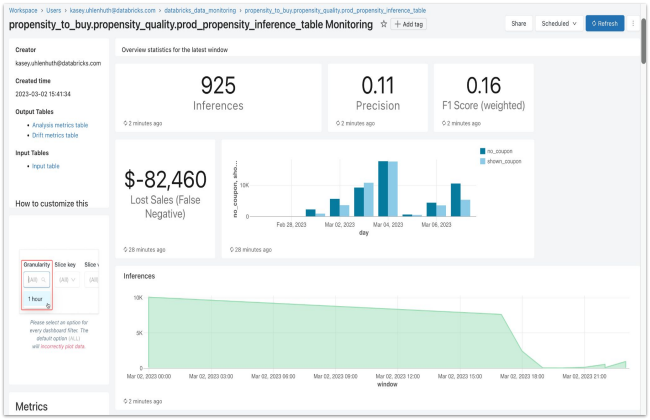
On one hand, it's essential to monitor data—whether it's input data, raw data, features, completions, index changes, or even shifts in embeddings and human feedback. Data always plays a crucial role in effective monitoring.

At the same time, it's important to keep an eye on AI assets themselves, like vanilla models and entire model pipelines, particularly if you're continuously training or running online evaluations. Tracking evaluation metrics is key to understanding model performance. And on top of that, you'll want a way to monitor how your operational costs are evolving as well.

Databricks Lakehouse Monitoring

Automated insights and out-of-the box metrics on data and ML pipelines

- **Fully managed** so no time wasted managing infrastructure, calculating metrics, or building dashboards from scratch
- **Frictionless** with easy setup and out-of-the-box metrics and generated dashboards
- **Unified** solution for data and models for holistic understanding



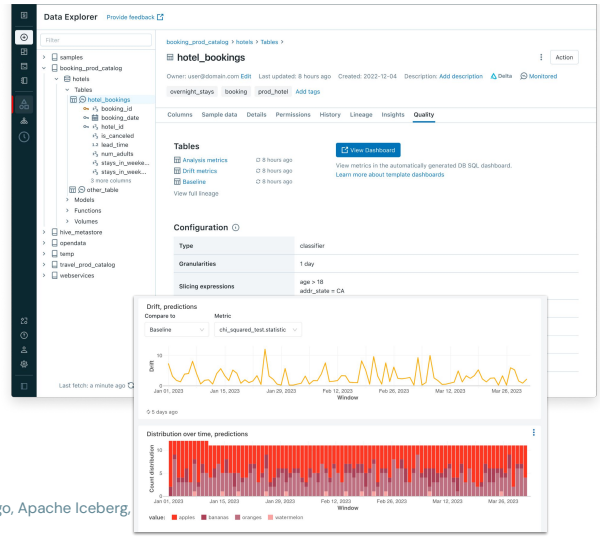
One of the main ways to meet these requirements is by using a built-in, data-centric lakehouse monitoring approach. This approach is fully managed, meaning there's no need to connect extra tools or move your data into another system. It's designed to be as non-intrusive as possible and stays focused on a specific table, so the process is smooth and simple. Customizing your monitoring is just a matter of making a straightforward API call or clicking a button. Plus, it's a unified solution, so it works not only for data quality monitoring, but also for tracking the behavior of an entire machine learning or generative AI system.

Built on Unity Catalog

Background service that incrementally processes data in Unity Catalog tables

For each monitored table:

- Calculates **profile metrics** table
- Calculates **drift metrics** table
- Supports **custom metrics** as SQL expressions
- **Auto-generates DBSQL dashboard** to visualize metrics over time
- Automatic PII detection
- Input expectations & rules

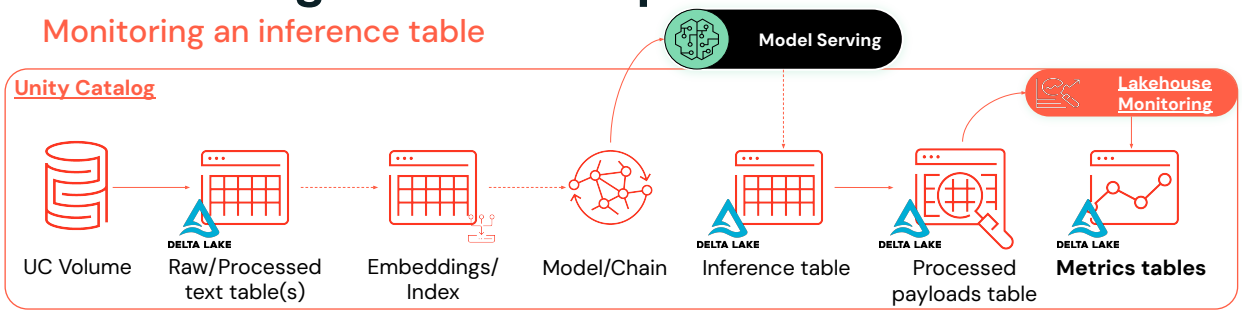


The starting point for this monitoring approach is having a table within a Unity Catalog schema, since the whole system is built as a Unity Catalog service. After setting up a monitor on the table, a backend monitoring job scans it and generates two new assets: a profile metrics table, which records distribution and statistical changes in the data, and a drift metrics table, which tracks changes over time using quantifiable measures like the population stability index or total variation distance. You can customize the monitor with your own SQL-based metrics.

Once those tables are created, Databricks automatically sets up a SQL dashboard built on top of queries from the profile and drift metrics tables. This lets you visually track how data is changing and spot trends at a glance. Beyond what's covered today, there are additional features like automatic PII detection and redaction, as well as setting field-level or row-level rules and expectations. If those rules are violated, the data can be quarantined, and violations are logged and tracked, so you're able to respond proactively.

Monitoring model's responses

Monitoring an inference table



Create quality monitoring of production data and models

1. **Unpack inference table**, calculate LLM-related evaluation metrics and materialize into processed table
   a. Schedule this as a triggered streaming job
2. **Enable Lakehouse Monitoring** on processed payloads table
3. Analyze dashboard and create SQL alerts on relevant calculated metrics

Once your inference table is set up for a large language model, the next step is to unpack the JSON payloads into structured columns like text inputs, outputs, and related metadata. During this process, you can also calculate key metrics, such as token counts, execution times, and toxicity or perplexity scores, then store these results in a new processed Delta table. This unpacking can run continuously as a streaming job, so all new requests and responses are automatically processed.

After the processed table is ready, Databricks' lakehouse monitoring service can be activated on it. This service creates additional tables for profiling and drift metrics, plus a SQL dashboard for visualizing trends. With this setup, you can configure proactive alerts on key metrics, helping you catch and address issues early.

Lakehouse Monitoring Workflows

A comprehensive monitoring tool for your data and AI system

Architecture Stages

- Development
  - Build into project within dev environment
- Testing
  - Develop integration tests that run a few iterations to ensure all monitoring is working
- Production
  - Deploy as part of the project to Production, including generating a baseline table
  - Write monitor tables to the prod catalog

Workflow Tips

1. Set up monitoring tables for all components
2. Model cost against performance
3. Refresh the tables and dashboard regularly
4. Set up key monitoring alerts
5. Connect rerun triggers to performance

A typical architecture follows three main stages: build in development, testing, and deploy in production. During development, models and data pipelines are created and configured. The testing stage ensures all components work reliably before production. Once deployed, monitoring tables are maintained in the production catalog to track ongoing performance and operational health.

To streamline the workflow, set up monitoring tables for critical components such as input data, knowledge data, human feedback, and model serving logs. Refresh these tables regularly to capture updated metrics and maintain visibility into system behavior. Track model performance across key metrics through automated dashboards, and configure monitoring alerts to quickly identify anomalies or performance drops.

Thank you for completing this lesson and continuing your journey to develop your skills with us.